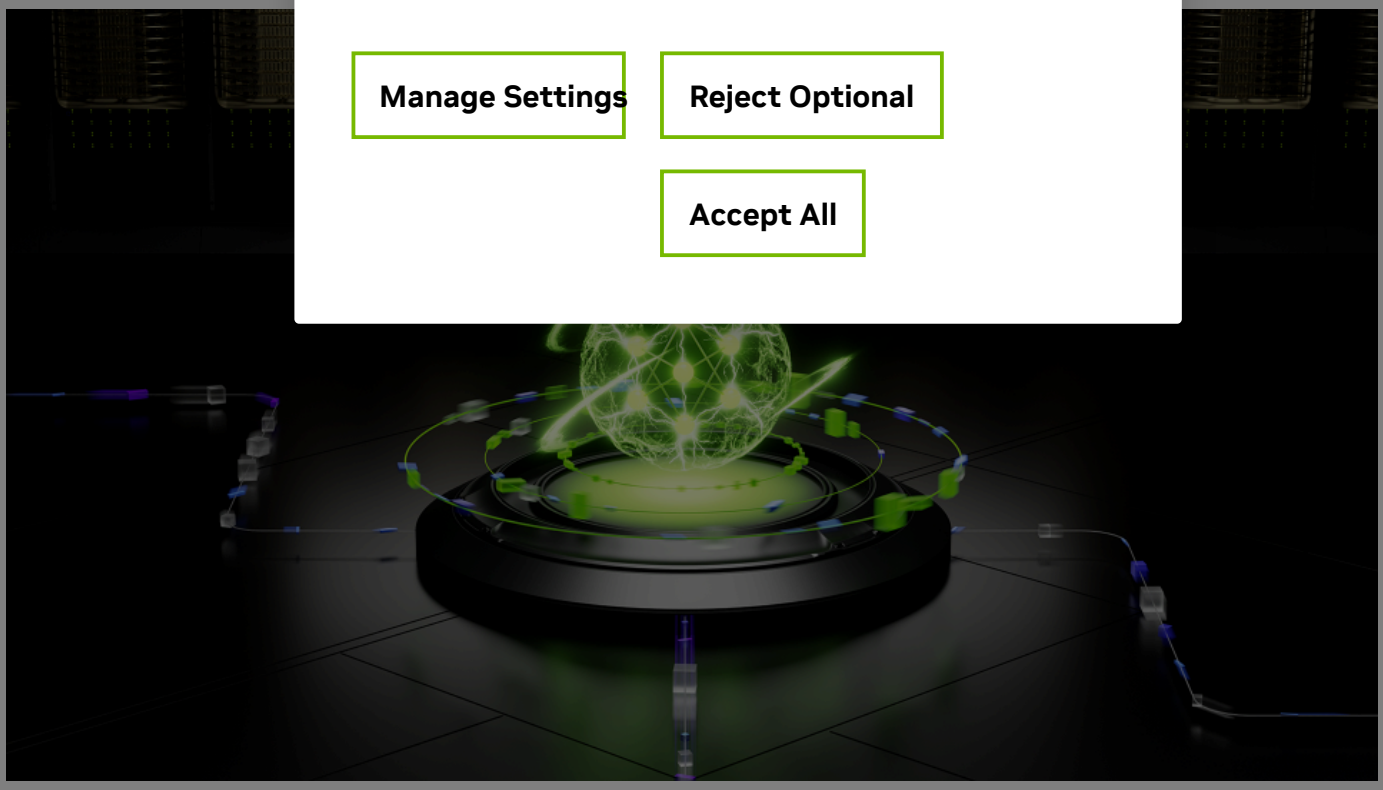
NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Manage Settings Reject Optional

Accept All

Full-St or Agenti DIA Dynam



Apr 17, 2026

+1 Like Discuss (0)

By Ishan Dhanani and Matej Kosec

Coding agents are starting to write production code at scale. Stripe's agents generate 1,300+ PRs per week. Ramp attributes 30% of merged PRs to agents. Spotify reports 650+ agent-generated PRs per month. Tools like Claude Code and Codex make hundreds of API calls per

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 1. Cumulative K *euse of prompt*

Lets take Claude Co sation prefix to KV cache, every s eams (or swarms) push this further with 97.2% aggregate cache hit rate across 4 Opus teammates. An 11.7x read/write ratio means the system reads from cache nearly 12 times for every token it writes. This is a write-once-read-many (WORM) access pattern: the system prompt and growing conversation prefix are computed once, then served from cache on every subsequent call. Maximizing cache reuse rate across all workers and keeping KV blocks warm and routable is the central optimization target for agentic inference.

These numbers come from managed API infrastructure where the provider controls prefix matching, cache placement, and eviction. For teams running open-source models on their own GPUs, none of this exists out of the box. We have been building Dynamo to close that gap. This post walks through how we are making Dynamo agent-native at three layers: the frontend API, the router, and KV cache management.

Throughout this post, we use three terms consistently:

- **Harness**: Agent framework that drives the workflow (Claude Code, Codex, OpenClaw, OpenCode, etc.)
- **Orchestrator**: Dynamo's routing, scheduling, and cache management layer

Layer 1: The frontend

Multi-proto

Agent harnesses are ... ing and tool calls. The key differe ... ssage content is a flat string and to ... how GLM and MiniMax API har ... behind the `v1/chat/completi` ... typed content blocks, so a ... s distinct objects. This matters ... ries, perform prompt optimization ... type. Dynamo serves all three endp ... eployment can act as the infere ... namo deployment of GLM-5 and MiniMax2.5 internally to power our Codex and Claude Code harnesses. This lets us benchmark our backend implementations against closed-source inference, targeting parity on cache reuse performance. We will be sharing a full write-up and some optimized recipes for deploying both models in the upcoming weeks.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA DEVELOPER Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

0:00 / 1:38

0:00 / 1:31

Video 2. Serving Codex with Dynamo

We have also invested in day-0 tool call and reasoning parsing support for various open-source models. If you find that a model is not supported, please open an issue or use the tool-call-parser-generator skill to generate it with your harness of choice.

Today, inference servers see anonymous tokenized requests. But agent harnesses have global context that the infrastructure never sees: which agents are blocked on tool calls, which just spawned, how many ck lookup or a long synthesis. Whe vidual token streams, so the orch hout affecting the end-user experie pauses. This is enough to optimiz

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 2. Agentic system stack comparison showing that with Dynamo, an nvext inference infrastructure layer is added to standardize access to GPU-backed runtimes across agent workflows.

Dynamo's new agent hints extension was designed to bridge this gap. It allows any harness to attach structured hints to a request across all three API endpoints, giving the router and runtime the context they need to make agent aware scheduling and caching decisions. This is a v1 API that we are actively co-designing with the community and would love feedback from teams building agent harnesses on what signals are most useful. Please reach out to us if you have any ideas or feedback.

```
  "tools": [...],
  "nvext": {
    "agent_hints":
      "osl": 256,
      "speculative
      "priority":
    },
    "cache_control
      "type": "eph
      "ttl": "1h"
    }
  }
}
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

The `agent_hints` fi

- **`priority`** control mean “more important” at the dering and backend-specific
- **`osl`** (output seque request will generate. The rout improves load balancing. A h gths per tool call type.
- **`speculative_prefill`** signals the orchestrator to begin caching this request’s prefix on a likely worker before the full request is ready. This is useful when the harness knows a tool call is about to return and wants to warm the cache ahead of time.

The `cache_control` field will look familiar to anyone who has used Anthropic’s prompt caching API. It tells the orchestrator to pin the computed prefix on the worker for the specified TTL, protecting it from eviction during tool call gaps. Currently `ephemeral` is the only supported type (to match Anthropic’s API). We discuss how this works in the cache retention section below. You can find complete documentation on agent hints here.

Layer 2: The router

A coding agent follows a sequential pattern: long prefill, tool call, extend prefix, repeat. A multi-agent harness fans out work across parallel subagents with short, independent contexts. Default round-robin routing is blind to both patterns — it cannot account for cache locality,

KV-Aware placement

Without cache-awar g on the
same worker as turn nt
performance bottler ntains a
global index of which st covers the
six iterations that go very request,
the router queries th hat
minimizes the comb ction is
tunable, and we show egies on top
of it.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Priority sch

`priority` is the sing portant" at
the Dynamo API leve

- At the **router**, higher-priority requests are shifted earlier in the queue when `--router-queue-threshold` is enabled.
- At the **engine**, Dynamo normalizes backend-specific polarity and forwards the request for queue ordering, preemption, and KV cache eviction.

At the router, incoming requests enter a `BinaryHeap<QueueEntry>` ordered by effective arrival time. A higher `priority` makes the request appear as if it arrived earlier, placing it ahead of lower-priority work. Requests only enter the queue when all workers exceed a configurable load threshold. Below that threshold, they bypass the queue entirely and go straight to worker selection. When capacity frees up (prefill completes or a request finishes), the queue drains highest-priority entries first.

Once dispatched, SGLang, vLLM, and TRT-LLM may interpret engine priority differently, so Dynamo normalizes the engine-facing value per backend. Engines like SGLang can also use priority-based radix cache eviction where lower-priority blocks are evicted first under memory pressure.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 3. This diagra here latency-sensitive tasks are t the engine.

Agentic workload routing strategies

A research agent with a 200K context window needs workers with enough free KV capacity to hold its full state. The router's default cost function (overlap score + decode load) handles the common case, but teams with domain-specific workloads can use the router's Python bindings to implement custom routing strategies. The core `KvRouter` class provides `best_worker()` for querying routing decisions, `get_potential_loads()` for per-worker load inspection, and `generate()` for routing + dispatch in one call. Custom routers register on the same service mesh as the default components and can override routing config per-request:

```
# Query per-worker load and overlap for custom routing logic
loads = await router.get_potential_loads(token_ids)
# Override routing config based on request properties
# Long contexts benefit from heavier overlap weighting
config = {"overlap_score_weight": 2.0} if len(token_ids) > 8192 else {}
worker_id, dp_rank, overlap = await router.best_worker(
    token_ids,
    request_id="req-123",
    update_indexer=True,
```

```
# has its own worker selection logic (e.g., session affinity)
stream = await router.generate(
    token_ids, model=model, worker_id=chosen_worker
)
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

The NeMo Agent Too ing agentic router. Their router e t to a Thompson Sampling best for which prefix pattern sured 4x reduction in p50 TTF of latency-sensitive requests a y pressure. See the NAT Dynam aking this available as a routing

Layer 3: K

Agentic workloads p pts reused every turn, reasoning all blocks identically. A 2-30 second tool call pause can age out an agent's entire prefix, forcing full recomputation when it resumes. The cache needs to understand block value, support cross-worker sharing, and respect agent lifecycle boundaries.

The problem with uniform eviction

Block Type	Reuse Pattern	Value
System prompt + tool definitions	Every turn	Highest
Conversation history	Subsequent turns, growing monotonically	High
Thinking/reasoning tokens	Typically zero reuse after reasoning loop closes (a significant portion of output)	Near-zero
Subagent KV	Multiple turns then agent dies. No need to retain	Near-zero

and when the agent resumes, the entire prefix must be recomputed. To solve this, we need to provide the orchestrator APIs to control which blocks should be retained, where they should live, and for how long.

KV cache as

Today, KV cache is tr s ~32K-token system prompt and hat serves its requests. When a lea itions, that shared prefix is reco our analysis of Claude Code team 4% cache hit rate vs. 91.3% for th), with the gap driven almost er al is to make high value KV cache e written once during cold sta

Solutions like SGLan ng toward a 4-tier memory hierarchy.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 4. Diagram sho *cal NVMe, and rem*

Blocks follow a write-through path: when a worker computes KV for a prefix, the blocks flow from GPU to CPU to disk automatically. Each block is deduplicated by sequence hash in a global registry. Once a block is registered, it is immutable and addressable by any worker that can reach the storage tier.

This directly solves the subagent cold-start problem. When the lead agent computes tool definitions and system prompt, those blocks write through to shared storage. When subagent 1 spawns on a different worker, the router queries the Flash Indexer, finds the blocks in shared storage, and the worker loads them via NIXL (RDMA read) instead of recomputing from scratch. Subagent 2 does the same. Four redundant prefill computations become one compute and three loads. The same mechanism addresses cache coherence in disaggregated prefill-decode serving. In disagg mode, the prefill worker computes KV and transfers it to the decode worker via NIXL. The decode worker generates tokens, producing new KV state. On the next turn, a prefill worker needs both the original prefix and the generated tokens from turn 1, but those live only on the decode worker. With shared storage, the decode worker writes its new blocks to the common tier and any prefill worker can fetch them on the next turn.

historical timing data to predict when an agent's tool call might return, which means it knows which blocks will be needed and when. We are building prefetch hooks so the harness can signal "bring these blocks f h the retention APIs (belov vent eviction, set priority to contro y are needed.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Figure 5. Diagram illustrating KV cache prefetch in agentic inference, where blocks are offloaded after an initial call and proactively prefetched from storage to GPU ahead of subsequent requests.

Selective cache retention

Making blocks globally available solves the sharing problem, but does not solve eviction. SGLang and vLLM both support priority-based eviction via a priority heap where the harness assigns a numeric priority per request and lower-priority blocks are evicted first. TensorRT-LLM takes this further with `TokenRangeRetentionConfig` (designed and implemented by a Dynamo team member@jthomson04) which allows per-region control within a single request.

unprioritized blocks (O(1), unchanged) and a priority queue for annotated blocks. The harness can express "system prompt blocks are evicted last (priority: 100); conversation context survives a 30-second tool cal out the engine needing to u

Anthropic's prompt ture. Dynamo's `cache_co` e. When a request includes `cac` router pins the matching prefix from eviction in the worker's L2 st

The next step is con directives apply to a single wor ext request routes to worker B, t HiCache/KVBM's sha ave it survive across workers: the ite-through path, and any worke n policy. Combined with the end lifecycle control across the full memory hierarchy.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Agent lifecycle awareness

Consider a typical Claude Code session. The lead agent runs for 20+ turns, accumulating a growing conversation prefix. Along the way it spawns explore subagents that each run 1-3 turns and terminate. It might spawn a team of 4 specialists that work in parallel on different subtasks and then terminate. Midway through, the agent hits a context limit and summarizes its history, compressing ~175K tokens down to ~40K. Each of these events produces ephemeral KV: blocks that will never be referenced again. Subagent termination, context summarization, and closed reasoning loops all generate ephemeral KV that occupies the same memory as high-value blocks like the system prompt. Reasoning models amplify this: `<think>...</think>` blocks account for ~40% of generated tokens but become ephemeral the moment the reasoning loop closes. Without lifecycle awareness, the cache treats all of these blocks identically.

The retention primitives from above (priority, TTL, token ranges) give us the building blocks. What is missing is th g a subagent's requests meral, the evictor can target th ly. When the subagent terminates ism applies to thinking tokens: t nd tag those blocks as ephemeral ormal blocks without any externa tagging, engine-native semar ctively exploring multiple di framework.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Closing t

The biggest optimiza harness knows and what the out to resume, which KV is exists at the harness layer but never crosses the API boundary. `nvext.agent_hints` is our first cut at closing that gap: a small set of structured signals that let the orchestrator make informed routing, scheduling, and cache management decisions instead of treating every request as anonymous tokens. This is a v1 API and we are actively evolving it. If you are building agent harnesses, running open-source models for agentic workloads, or thinking about cache-aware inference, we want to hear what signals matter most for your use case. Reach out on GitHub or tag us on X: @0xishand, @KranenKyle, @flowpow123.

Discuss (0) +1 Like

Tags

Agentic AI / Generative AI | Data Center / Cloud | Developer Tools & Techniques | General | Dynamo | Intermediate Technical | Tutorial | Build AI Agents

About the Authors

background in applied statistic and his work focuses primarily building scalable, distributed inference systems.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Comments

Start the d

Related posts

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Scaling Autonomous AI Agents and Workloads with NVIDIA DGX Spark

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA Vera C

for AI Factori

fficiency

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA Dynamo Adds Support for AWS Services to Deliver Cost-Efficient Inference at Scale

The Next Generation of AI Begins

It all starts here. June 1–4 | Taipei

Register Now

NVIDIA GTC

Sign up for NVIDIA News

Subscribe

Follow NVIDIA Developer

NVIDIA DEVELOPER

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.